

ADO.NET Interview Questions and Answers

Table of Contents

1. [Basic Level Questions](#)
 2. [Intermediate Level Questions](#)
 3. [Advanced Level Questions](#)
 4. [Practical Coding Examples](#)
-

Basic Level Questions

1. What is ADO. NET and what are its main components?

Answer:

[ADO.NET](#) (ActiveX Data Objects . NET) is a data access technology from Microsoft that provides a bridge between the front-end controls and the back-end database.

Main Components:

1. Data Provider Components (Connected Architecture):

- Connection - Establishes connection to database
- Command - Executes SQL commands
- DataReader - Forward-only, read-only data stream
- DataAdapter - Bridge between DataSet and database

2. DataSet Components (Disconnected Architecture):

- DataSet - In-memory cache of data
- DataTable - Represents a table
- DataRow - Represents a row
- DataColumn - Represents a column
- DataRelation - Represents relationship between tables

// Architecture Overview

Application Layer	
Connected (DataReader)	Disconnected (DataSet)
Data Provider (SqlClient, etc.)	
Database (SQL Server, Oracle, etc.)	

2. What is the difference between connected and disconnected architecture?

Answer:

Feature	Connected Architecture	Disconnected Architecture
Connection	Keeps connection open	Opens only when needed
Objects Used	Command, DataReader	DataAdapter, DataSet
Performance	Faster for read-only	Better for multiple operations
Memory	Less memory usage	More memory usage
Scalability	Less scalable	More scalable
Use Case	Real-time data	Offline data manipulation
Concurrency	Lower	Higher

```
// Connected Architecture
using (SqlConnection conn = new SqlConnection(connectionString))
{
    conn.Open();
    SqlCommand cmd = new SqlCommand("SELECT * FROM Products", conn);
    SqlDataReader reader = cmd.ExecuteReader();

    while (reader.Read())
    {
        Console.WriteLine(reader["ProductName"]);
    }
    // Connection remains open until closed
}

// Disconnected Architecture
using (SqlConnection conn = new SqlConnection(connectionString))
{
    SqlDataAdapter adapter = new SqlDataAdapter("SELECT * FROM Products", conn);
    DataSet ds = new DataSet();
    adapter.Fill(ds, "Products");
    // Connection automatically closed after Fill

    // Work with data offline
    foreach (DataRow row in ds.Tables["Products"].Rows)
    {
        Console.WriteLine(row["ProductName"]);
    }
}
```

3. What are the different Execute methods in ADO.NET Command object?**Answer:**

```
SqlCommand cmd = new SqlCommand();

// 1. ExecuteReader() - Returns DataReader for SELECT queries
SqlDataReader reader = cmd.ExecuteReader();
// Use: SELECT queries that return multiple rows

// 2. ExecuteScalar() - Returns first column of first row
object result = cmd.ExecuteScalar();
// Use: Aggregate functions (COUNT, SUM, MAX, etc.)

// 3. ExecuteNonQuery() - Returns number of rows affected
int rowsAffected = cmd.ExecuteNonQuery();
// Use: INSERT, UPDATE, DELETE, CREATE, ALTER
```

```
// 4. ExecuteXmlReader() - Returns XmlReader
XmlReader xmlReader = cmd.ExecuteXmlReader();
// Use: SQL Server XML queries (FOR XML)
```

Examples:

```
// ExecuteScalar Example
string query = "SELECT COUNT(*) FROM Products";
SqlCommand cmd = new SqlCommand(query, connection);
int count = (int)cmd.ExecuteScalar();

// ExecuteNonQuery Example
string query = "DELETE FROM Products WHERE ProductID = @id";
SqlCommand cmd = new SqlCommand(query, connection);
cmd.Parameters.AddWithValue("@id", 101);
int rowsDeleted = cmd.ExecuteNonQuery();

// ExecuteReader Example
string query = "SELECT * FROM Products";
SqlCommand cmd = new SqlCommand(query, connection);
SqlDataReader reader = cmd.ExecuteReader();
while (reader.Read())
{
    Console.WriteLine($"{reader["ProductID"]}: {reader["ProductName"]}");
}
```

4. What is SqlDataReader and what are its characteristics?

Answer:

SqlDataReader is a forward-only, read-only stream of data from a database.

Characteristics:

- Read-only (cannot modify data)
- Forward-only (cannot go backwards)
- Connection-oriented (keeps connection open)
- Fastest way to read data
- Firehose cursor
- Must be explicitly closed

```
using (SqlConnection conn = new SqlConnection(connectionString))
{
    conn.Open();
    SqlCommand cmd = new SqlCommand("SELECT * FROM Employees", conn);

    SqlDataReader reader = cmd.ExecuteReader();

    // Check if data exists
    if (reader.HasRows)
    {
        while (reader.Read())
        {
            // Access by column name
            string name = reader["EmployeeName"].ToString();

            // Access by index
```

```
        int id = reader.GetInt32(0);

        // Safe access with null check
        string email = reader.IsDBNull(3) ? "" : reader.GetString(3);

        Console.WriteLine($"ID: {id}, Name: {name}, Email: {email}");
    }
}

reader.Close();
}
```

5. What is a DataSet and how is it different from DataReader?

Answer:

DataSet is an in-memory representation of data that can contain multiple DataTables with relationships.

Feature	DataSet	DataReader
Architecture	Disconnected	Connected
Navigation	Forward and backward	Forward-only
Read/Write	Can update data	Read-only
Memory	More memory	Less memory
Speed	Slower	Faster
Relationships	Supports relations	No relations
Multiple Tables	Yes	One at a time
Serialization	Serializable (XML)	Not serializable

```
// DataSet Example
DataSet ds = new DataSet("MyDataSet");

using (SqlConnection conn = new SqlConnection(connectionString))
{
    // Load Products
    SqlDataAdapter productAdapter = new SqlDataAdapter(
        "SELECT * FROM Products", conn);
    productAdapter.Fill(ds, "Products");

    // Load Categories
    SqlDataAdapter categoryAdapter = new SqlDataAdapter(
        "SELECT * FROM Categories", conn);
    categoryAdapter.Fill(ds, "Categories");

    // Create relationship
    DataRelation relation = new DataRelation(
        "ProductCategory",
        ds.Tables["Categories"].Columns["CategoryID"],
        ds.Tables["Products"].Columns["CategoryID"]
    );
    ds.Relations.Add(relation);
}

// Access data offline
foreach (DataRow catRow in ds.Tables["Categories"].Rows)
{
    Console.WriteLine($"Category: {catRow["CategoryName"]}");
}
```

```
// Get related products
DataRow[] products = catRow.GetChildRows("ProductCategory");
foreach (DataRow prodRow in products)
{
    Console.WriteLine($" - {prodRow["ProductName"]}");
}
}
```

Intermediate Level Questions

6. What is DataAdapter and how does it work?

Answer:

DataAdapter serves as a bridge between DataSet and the database. It uses four commands internally.

Four Internal Commands:

1. SelectCommand - Retrieves data
2. InsertCommand - Inserts new records
3. UpdateCommand - Updates existing records
4. DeleteCommand - Deletes records

```
using (SqlConnection conn = new SqlConnection(connectionString))
{
    // Create DataAdapter
    SqlDataAdapter adapter = new SqlDataAdapter();

    // Set SelectCommand
    adapter.SelectCommand = new SqlCommand(
        "SELECT * FROM Products", conn);

    // Set InsertCommand
    adapter.InsertCommand = new SqlCommand(
        "INSERT INTO Products (ProductName, Price) VALUES (@name, @price)", conn);
    adapter.InsertCommand.Parameters.Add("@name", SqlDbType.NVarChar, 50, "ProductName");
    adapter.InsertCommand.Parameters.Add("@price", SqlDbType.Decimal, 0, "Price");

    // Set UpdateCommand
    adapter.UpdateCommand = new SqlCommand(
        "UPDATE Products SET ProductName=@name, Price=@price WHERE ProductID=@id", conn);
    adapter.UpdateCommand.Parameters.Add("@name", SqlDbType.NVarChar, 50, "ProductName");
    adapter.UpdateCommand.Parameters.Add("@price", SqlDbType.Decimal, 0, "Price");
    adapter.UpdateCommand.Parameters.Add("@id", SqlDbType.Int, 0, "ProductID");

    // Set DeleteCommand
    adapter.DeleteCommand = new SqlCommand(
        "DELETE FROM Products WHERE ProductID=@id", conn);
    adapter.DeleteCommand.Parameters.Add("@id", SqlDbType.Int, 0, "ProductID");

    // Fill DataSet
    DataSet ds = new DataSet();
    adapter.Fill(ds, "Products");

    // Modify data
    DataTable table = ds.Tables["Products"];
}
```

```

        table.Rows[0]["ProductName"] = "Updated Product";
        table.Rows.Add(null, "New Product", 99.99);
        table.Rows[1].Delete();

        // Update database
        adapter.Update(ds, "Products");
    }

```

Using SqlCommandBuilder (Automatic):

```

SqlDataAdapter adapter = new SqlDataAdapter(
    "SELECT * FROM Products", connection);

// Automatically generates Insert, Update, Delete commands
SqlCommandBuilder builder = new SqlCommandBuilder(adapter);

DataSet ds = new DataSet();
adapter.Fill(ds, "Products");

// Make changes
ds.Tables["Products"].Rows[0]["Price"] = 150.00;

// Update automatically handles Insert/Update/Delete
adapter.Update(ds, "Products");

```

7. How do you implement parameterized queries to prevent SQL Injection?

Answer:

Parameterized queries prevent SQL injection by treating user input as data, not executable code.

```

// BAD - Vulnerable to SQL Injection
public DataTable GetProducts_Unsafe(string category)
{
    string query = "SELECT * FROM Products WHERE Category = '" + category + "'";
    // If category = ''; DROP TABLE Products; --"
    // Query becomes: SELECT * FROM Products WHERE Category = ''; DROP TABLE Products; --"
}

// GOOD - Using Parameters
public DataTable GetProducts_Safe(string category)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        string query = "SELECT * FROM Products WHERE Category = @category";
        SqlCommand cmd = new SqlCommand(query, conn);

        // Add parameter
        cmd.Parameters.AddWithValue("@category", category);

        SqlDataAdapter adapter = new SqlDataAdapter(cmd);
        DataTable dt = new DataTable();
        adapter.Fill(dt);

        return dt;
    }
}

// BETTER - Using SqlParameter with explicit type
public void InsertProduct(string name, decimal price, int categoryId)

```

```

{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        string query = @"INSERT INTO Products (ProductName, Price, CategoryID)
                        VALUES (@name, @price, @categoryId)";

        SqlCommand cmd = new SqlCommand(query, conn);

        // Explicitly define parameter types
        cmd.Parameters.Add("@name", SqlDbType.NVarChar, 100).Value = name;
        cmd.Parameters.Add("@price", SqlDbType.Decimal).Value = price;
        cmd.Parameters.Add("@categoryId", SqlDbType.Int).Value = categoryId;

        conn.Open();
        cmd.ExecuteNonQuery();
    }
}

// Multiple parameters example
public DataTable SearchProducts(string searchTerm, decimal minPrice, decimal maxPrice)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        string query = @"SELECT * FROM Products
                        WHERE ProductName LIKE @search
                        AND Price BETWEEN @minPrice AND @maxPrice";

        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddWithValue("@search", "%" + searchTerm + "%");
        cmd.Parameters.AddWithValue("@minPrice", minPrice);
        cmd.Parameters.AddWithValue("@maxPrice", maxPrice);

        SqlDataAdapter adapter = new SqlDataAdapter(cmd);
        DataTable dt = new DataTable();
        adapter.Fill(dt);

        return dt;
    }
}

```

8. What are stored procedures and how do you execute them in [ADO.NET](#)?

Answer:

Stored procedures are precompiled SQL statements stored in the database.

Benefits:

- Better performance (precompiled)
- Reduced network traffic
- Enhanced security
- Code reusability
- Easier maintenance

-- SQL Server Stored Procedure Examples

-- Simple procedure

```

CREATE PROCEDURE GetAllProducts
AS

```

```
BEGIN
    SELECT * FROM Products
END

-- With INPUT parameters
CREATE PROCEDURE GetProductsByCategory
    @CategoryID INT
AS
BEGIN
    SELECT * FROM Products WHERE CategoryID = @CategoryID
END

-- With OUTPUT parameters
CREATE PROCEDURE InsertProduct
    @ProductName NVARCHAR(100),
    @Price DECIMAL(10,2),
    @CategoryID INT,
    @NewProductID INT OUTPUT
AS
BEGIN
    INSERT INTO Products (ProductName, Price, CategoryID)
    VALUES (@ProductName, @Price, @CategoryID)

    SET @NewProductID = SCOPE_IDENTITY()
END

-- With RETURN value
CREATE PROCEDURE GetProductCount
AS
BEGIN
    DECLARE @Count INT
    SELECT @Count = COUNT(*) FROM Products
    RETURN @Count
END

// Execute stored procedure - No parameters
public DataTable GetAllProducts()
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        SqlCommand cmd = new SqlCommand("GetAllProducts", conn);
        cmd.CommandType = CommandType.StoredProcedure;

        SqlDataAdapter adapter = new SqlDataAdapter(cmd);
        DataTable dt = new DataTable();
        adapter.Fill(dt);

        return dt;
    }
}

// Execute stored procedure - INPUT parameters
public DataTable GetProductsByCategory(int categoryId)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        SqlCommand cmd = new SqlCommand("GetProductsByCategory", conn);
        cmd.CommandType = CommandType.StoredProcedure;

        cmd.Parameters.AddWithValue("@CategoryID", categoryId);

        SqlDataAdapter adapter = new SqlDataAdapter(cmd);
```

```
        DataTable dt = new DataTable();
        adapter.Fill(dt);

        return dt;
    }
}

// Execute stored procedure - OUTPUT parameters
public int InsertProduct(string name, decimal price, int categoryId)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        SqlCommand cmd = new SqlCommand("InsertProduct", conn);
        cmd.CommandType = CommandType.StoredProcedure;

        // Input parameters
        cmd.Parameters.AddWithValue("@ProductName", name);
        cmd.Parameters.AddWithValue("@Price", price);
        cmd.Parameters.AddWithValue("@CategoryId", categoryId);

        // Output parameter
        SqlParameter outputParam = new SqlParameter("@NewProductID", SqlDbType.Int);
        outputParam.Direction = ParameterDirection.Output;
        cmd.Parameters.Add(outputParam);

        conn.Open();
        cmd.ExecuteNonQuery();

        // Get output value
        int newProductId = (int)outputParam.Value;
        return newProductId;
    }
}

// Execute stored procedure - RETURN value
public int GetProductCount()
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        SqlCommand cmd = new SqlCommand("GetProductCount", conn);
        cmd.CommandType = CommandType.StoredProcedure;

        // Return parameter
        SqlParameter returnParam = new SqlParameter();
        returnParam.Direction = ParameterDirection.ReturnValue;
        cmd.Parameters.Add(returnParam);

        conn.Open();
        cmd.ExecuteNonQuery();

        int count = (int)returnParam.Value;
        return count;
    }
}
```

9. What is Transaction in ADO.NET and how do you implement it?

Answer:

A transaction is a unit of work that either completely succeeds or completely fails (ACID properties).

ACID Properties:

- **Atomicity** - All or nothing
- **Consistency** - Data remains valid
- **Isolation** - Transactions don't interfere
- **Durability** - Changes are permanent

```
// Basic Transaction
public void TransferMoney(int fromAccount, int toAccount, decimal amount)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        conn.Open();
        SqlTransaction transaction = conn.BeginTransaction();

        try
        {
            // Debit from account
            SqlCommand debitCmd = new SqlCommand(
                "UPDATE Accounts SET Balance = Balance - @amount WHERE AccountID = @id",
                conn, transaction);
            debitCmd.Parameters.AddWithValue("@amount", amount);
            debitCmd.Parameters.AddWithValue("@id", fromAccount);
            debitCmd.ExecuteNonQuery();

            // Credit to account
            SqlCommand creditCmd = new SqlCommand(
                "UPDATE Accounts SET Balance = Balance + @amount WHERE AccountID = @id",
                conn, transaction);
            creditCmd.Parameters.AddWithValue("@amount", amount);
            creditCmd.Parameters.AddWithValue("@id", toAccount);
            creditCmd.ExecuteNonQuery();

            // Commit if both succeed
            transaction.Commit();
            Console.WriteLine("Transaction completed successfully");
        }
        catch (Exception ex)
        {
            // Rollback if any fails
            transaction.Rollback();
            Console.WriteLine($"Transaction failed: {ex.Message}");
            throw;
        }
    }
}

// Transaction with Isolation Level
public void TransactionWithIsolationLevel()
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        conn.Open();

        // Set isolation level
        SqlTransaction transaction = conn.BeginTransaction(
            IsolationLevel.ReadCommitted);

        try
        {
            SqlCommand cmd = new SqlCommand(
```

```

        "INSERT INTO Orders (OrderDate, CustomerID) VALUES (@date, @custId)",
        conn, transaction);
cmd.Parameters.AddWithValue("@date", DateTime.Now);
cmd.Parameters.AddWithValue("@custId", 1);
cmd.ExecuteNonQuery();

    transaction.Commit();
}
catch
{
    transaction.Rollback();
    throw;
}
}

// TransactionScope (Distributed Transaction)
using System.Transactions;

public void TransactionScopeExample()
{
    using (TransactionScope scope = new TransactionScope())
    {
        try
        {
            using (SqlConnection conn1 = new SqlConnection(connectionString1))
            {
                conn1.Open();
                SqlCommand cmd1 = new SqlCommand(
                    "INSERT INTO Orders (OrderDate) VALUES (@date)", conn1);
                cmd1.Parameters.AddWithValue("@date", DateTime.Now);
                cmd1.ExecuteNonQuery();
            }

            using (SqlConnection conn2 = new SqlConnection(connectionString2))
            {
                conn2.Open();
                SqlCommand cmd2 = new SqlCommand(
                    "INSERT INTO AuditLog (Action) VALUES (@action)", conn2);
                cmd2.Parameters.AddWithValue("@action", "Order Created");
                cmd2.ExecuteNonQuery();
            }

            // Complete the transaction
            scope.Complete();
        }
        catch (Exception ex)
        {
            // Automatic rollback if Complete() not called
            Console.WriteLine($"Transaction failed: {ex.Message}");
        }
    }
}

```

Isolation Levels:

```

// Different Isolation Levels
public enum IsolationLevel
{
    ReadUncommitted, // Dirty reads possible
    ReadCommitted,   // Default, no dirty reads
    RepeatableRead,  // No dirty/non-repeatable reads
}

```

```
    Serializable,    // Most restrictive, no phantom reads
    Snapshot         // Uses row versioning
}
```

10. What is Connection Pooling and how does it work?

Answer:

Connection pooling is a technique that reuses existing database connections instead of creating new ones.

Benefits:

- Improved performance
- Reduced overhead
- Better resource management
- Increased scalability

```
// Connection pooling is automatic in ADO.NET
// Controlled via connection string parameters

// Connection string with pooling parameters
string connectionString =
    "Server=myServer;" +
    "Database=myDB;" +
    "User Id=myUser;" +
    "Password=myPass;" +
    "Pooling=true;" +                // Enable pooling (default: true)
    "Min Pool Size=5;" +             // Minimum connections (default: 0)
    "Max Pool Size=100;" +           // Maximum connections (default: 100)
    "Connection Lifetime=0;" +       // Max lifetime in seconds (default: 0 = no limit)
    "Connection Reset=true;" +       // Reset connection when returned (default: true)

// Pooling in action
public void DemonstratePooling()
{
    // First connection - creates new physical connection
    using (SqlConnection conn1 = new SqlConnection(connectionString))
    {
        conn1.Open();
        Console.WriteLine("Connection 1 opened");
    } // Connection returned to pool, not closed

    // Second connection - reuses connection from pool
    using (SqlConnection conn2 = new SqlConnection(connectionString))
    {
        conn2.Open();
        Console.WriteLine("Connection 2 opened (reused from pool)");
    }
}

// Clear connection pool
public void ClearPoolExample()
{
    SqlConnection conn = new SqlConnection(connectionString);

    // Clear pool for specific connection
    SqlConnection.ClearPool(conn);

    // Clear all pools
}
```

```
        SqlConnection.ClearAllPools();
    }

    // Disable pooling for specific connection
    string noPoolConnectionString =
        "Server=myServer;Database=myDB;User Id=myUser;Password=myPass;Pooling=false;";
```

Advanced Level Questions

11. How do you handle multiple result sets in [ADO.NET](#)?

Answer:

```
// SQL that returns multiple result sets
string query = @"
    SELECT * FROM Customers;
    SELECT * FROM Orders;
    SELECT * FROM Products;
";

public void HandleMultipleResultSets()
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        SqlCommand cmd = new SqlCommand(query, conn);
        conn.Open();

        SqlDataReader reader = cmd.ExecuteReader();

        // First result set - Customers
        Console.WriteLine("=== Customers ===");
        while (reader.Read())
        {
            Console.WriteLine($"{reader["CustomerID"]}: {reader["CustomerName"]}");
        }

        // Move to next result set
        if (reader.NextResult())
        {
            Console.WriteLine("\n=== Orders ===");
            while (reader.Read())
            {
                Console.WriteLine($"{reader["OrderID"]}: {reader["OrderDate"]}");
            }
        }

        // Move to third result set
        if (reader.NextResult())
        {
            Console.WriteLine("\n=== Products ===");
            while (reader.Read())
            {
                Console.WriteLine($"{reader["ProductID"]}: {reader["ProductName"]}");
            }
        }

        reader.Close();
    }
}
```

```
// Using DataSet with DataAdapter
public DataSet GetMultipleTablesDataSet()
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        SqlDataAdapter adapter = new SqlDataAdapter(query, conn);
        DataSet ds = new DataSet();

        adapter.Fill(ds);

        // Access different tables
        DataTable customers = ds.Tables[0];
        DataTable orders = ds.Tables[1];
        DataTable products = ds.Tables[2];

        // Rename tables for clarity
        customers.TableName = "Customers";
        orders.TableName = "Orders";
        products.TableName = "Products";

        return ds;
    }
}
```

12. What is the difference between DataView and DataTable?

Answer:

```
// DataTable - Storage
DataTable dt = new DataTable("Products");
dt.Columns.Add("ProductID", typeof(int));
dt.Columns.Add("ProductName", typeof(string));
dt.Columns.Add("Price", typeof(decimal));
dt.Columns.Add("InStock", typeof(bool));

dt.Rows.Add(1, "Laptop", 1200.00, true);
dt.Rows.Add(2, "Mouse", 25.00, false);
dt.Rows.Add(3, "Keyboard", 75.00, true);
dt.Rows.Add(4, "Monitor", 300.00, true);

// DataView - View/Filter of DataTable
DataView dv = new DataView(dt);

// Filter
dv.RowFilter = "InStock = true AND Price > 50";

// Sort
dv.Sort = "Price DESC";

// Display filtered and sorted data
foreach (DataRowView row in dv)
{
    Console.WriteLine($"{row["ProductName"]}: ${row["Price"]}");
}
// Output:
// Monitor: $300.00
// Keyboard: $75.00

// Multiple views on same table
DataView cheapItems = new DataView(dt);
```

```
cheapItems.RowFilter = "Price < 100";

DataView expensiveItems = new DataView(dt);
expensiveItems.RowFilter = "Price >= 100";

// Search in DataView
dv.Sort = "ProductID";
int index = dv.Find(2); // Find by sorted column
if (index >= 0)
{
    DataRowView row = dv[index];
    Console.WriteLine($"Found: {row["ProductName"]}");
}

// DataView Row States
DataView newRowsView = new DataView(dt);
newRowsView.RowStateFilter = DataRowState.Added;

DataView modifiedView = new DataView(dt);
modifiedView.RowStateFilter = DataRowState.ModifiedCurrent;
```

13. How do you implement Bulk Copy operations?

Answer:

SqlBulkCopy is used for efficiently loading large amounts of data into SQL Server tables.

```
// Bulk insert from DataTable
public void BulkInsertFromDataTable(DataTable sourceData)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        conn.Open();

        using (SqlBulkCopy bulkCopy = new SqlBulkCopy(conn))
        {
            // Set destination table
            bulkCopy.DestinationTableName = "Products";

            // Set batch size
            bulkCopy.BatchSize = 1000;

            // Set timeout
            bulkCopy.BulkCopyTimeout = 60;

            // Map columns (if names don't match)
            bulkCopy.ColumnMappings.Add("SourceProductID", "ProductID");
            bulkCopy.ColumnMappings.Add("SourceName", "ProductName");
            bulkCopy.ColumnMappings.Add("SourcePrice", "Price");

            // Event handlers
            bulkCopy.SqlRowsCopied += (sender, e) =>
            {
                Console.WriteLine($"Rows copied: {e.RowsCopied}");
            };
            bulkCopy.NotifyAfter = 1000;

            try
            {
                bulkCopy.WriteToServer(sourceData);
                Console.WriteLine("Bulk insert completed successfully");
            }
        }
    }
}
```

```
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error: {ex.Message}");
        }
    }
}

// Bulk insert from DataReader
public void BulkInsertFromDataReader()
{
    using (SqlConnection sourceConn = new SqlConnection(sourceConnectionString))
    using (SqlConnection destConn = new SqlConnection(destConnectionString))
    {
        sourceConn.Open();
        destConn.Open();

        SqlCommand cmd = new SqlCommand("SELECT * FROM Products", sourceConn);
        SqlDataReader reader = cmd.ExecuteReader();

        using (SqlBulkCopy bulkCopy = new SqlBulkCopy(destConn))
        {
            bulkCopy.DestinationTableName = "Products";
            bulkCopy.BatchSize = 5000;

            bulkCopy.WriteToServer(reader);
        }

        reader.Close();
    }
}

// Bulk insert with transaction
public void BulkInsertWithTransaction(DataTable data)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        conn.Open();
        SqlTransaction transaction = conn.BeginTransaction();

        try
        {
            using (SqlBulkCopy bulkCopy = new SqlBulkCopy(conn,
                SqlBulkCopyOptions.Default, transaction))
            {
                bulkCopy.DestinationTableName = "Products";
                bulkCopy.WriteToServer(data);
            }

            transaction.Commit();
            Console.WriteLine("Bulk insert committed");
        }
        catch (Exception ex)
        {
            transaction.Rollback();
            Console.WriteLine($"Bulk insert rolled back: {ex.Message}");
        }
    }
}

// Bulk insert from CSV file
```

```
public void BulkInsertFromCSV(string filePath)
{
    DataTable dt = new DataTable();
    dt.Columns.Add("ProductName", typeof(string));
    dt.Columns.Add("Price", typeof(decimal));
    dt.Columns.Add("CategoryID", typeof(int));

    // Read CSV
    foreach (string line in File.ReadLines(filePath).Skip(1)) // Skip header
    {
        string[] parts = line.Split(',');
        dt.Rows.Add(parts[0], decimal.Parse(parts[1]), int.Parse(parts[2]));
    }

    // Bulk insert
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        conn.Open();
        using (SqlBulkCopy bulkCopy = new SqlBulkCopy(conn))
        {
            bulkCopy.DestinationTableName = "Products";
            bulkCopy.WriteToServer(dt);
        }
    }
}
```

14. How do you implement asynchronous operations in [ADO.NET](#)?

Answer:

```
// Async ExecuteReader
public async Task<List<Product>> GetProductsAsync()
{
    List<Product> products = new List<Product>();

    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        await conn.OpenAsync();

        SqlCommand cmd = new SqlCommand("SELECT * FROM Products", conn);

        using (SqlDataReader reader = await cmd.ExecuteReaderAsync())
        {
            while (await reader.ReadAsync())
            {
                products.Add(new Product
                {
                    ProductID = reader.GetInt32(0),
                    ProductName = reader.GetString(1),
                    Price = reader.GetDecimal(2)
                });
            }
        }

        return products;
    }
}

// Async ExecuteNonQuery
public async Task<int> InsertProductAsync(Product product)
{
}
```

```
using (SqlConnection conn = new SqlConnection(connectionString))
{
    await conn.OpenAsync();

    SqlCommand cmd = new SqlCommand(
        "INSERT INTO Products (ProductName, Price) VALUES (@name, @price)",
        conn);
    cmd.Parameters.AddWithValue("@name", product.ProductName);
    cmd.Parameters.AddWithValue("@price", product.Price);

    int rowsAffected = await cmd.ExecuteNonQueryAsync();
    return rowsAffected;
}

// Async ExecuteScalar
public async Task<int> GetProductCountAsync()
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        await conn.OpenAsync();

        SqlCommand cmd = new SqlCommand("SELECT COUNT(*) FROM Products", conn);

        object result = await cmd.ExecuteScalarAsync();
        return Convert.ToInt32(result);
    }
}

// Parallel async operations
public async Task ProcessMultipleQueriesAsync()
{
    Task<List<Product>> productsTask = GetProductsAsync();
    Task<int> countTask = GetProductCountAsync();
    Task<List<Category>> categoriesTask = GetCategoriesAsync();

    // Wait for all to complete
    await Task.WhenAll(productsTask, countTask, categoriesTask);

    List<Product> products = await productsTask;
    int count = await countTask;
    List<Category> categories = await categoriesTask;

    Console.WriteLine($"Loaded {products.Count} products, " +
        $"{count} total count, " +
        $"{categories.Count} categories");
}

// Async with cancellation token
public async Task<List<Product>> GetProductsAsync(Cancellation_token cancellation_token)
{
    List<Product> products = new List<Product>();

    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        await conn.OpenAsync(cancellation_token);

        SqlCommand cmd = new SqlCommand("SELECT * FROM Products", conn);

        using (SqlDataReader reader = await cmd.ExecuteReaderAsync(cancellation_token))
        {
            while (await reader.ReadAsync(cancellation_token))
            {
                products.Add(new Product { ProductName = reader.GetString(0), Price = reader.GetDecimal(1) });
            }
        }
    }
}
```

```

        {
            products.Add(new Product
            {
                ProductID = reader.GetInt32(0),
                ProductName = reader.GetString(1)
            });
        }
    }

    return products;
}

// Usage with cancellation
public async Task UsageExample()
{
    CancellationTokenSource cts = new CancellationTokenSource();
    cts.CancelAfter(TimeSpan.FromSeconds(5)); // Cancel after 5 seconds

    try
    {
        List<Product> products = await GetProductsAsync(cts.Token);
        Console.WriteLine($"Retrieved {products.Count} products");
    }
    catch (OperationCanceledException)
    {
        Console.WriteLine("Operation was cancelled");
    }
}

```

15. What are Table-Valued Parameters and how do you use them?

Answer:

Table-Valued Parameters (TVPs) allow passing multiple rows of data to stored procedures efficiently.

```

-- Step 1: Create User-Defined Table Type
CREATE TYPE ProductTableType AS TABLE
(
    ProductName NVARCHAR(100),
    Price DECIMAL(10,2),
    CategoryID INT
)

-- Step 2: Create Stored Procedure
CREATE PROCEDURE InsertMultipleProducts
    @Products ProductTableType READONLY
AS
BEGIN
    INSERT INTO Products (ProductName, Price, CategoryID)
    SELECT ProductName, Price, CategoryID
    FROM @Products
END

// C# Implementation
public void InsertMultipleProducts(List<Product> products)
{
    // Create DataTable matching table type structure
    DataTable productTable = new DataTable();
    productTable.Columns.Add("ProductName", typeof(string));
    productTable.Columns.Add("Price", typeof(decimal));
}

```

```
productTable.Columns.Add("CategoryID", typeof(int));

// Fill DataTable with product data
foreach (var product in products)
{
    productTable.Rows.Add(
        product.ProductName,
        product.Price,
        product.CategoryID
    );
}

using (SqlConnection conn = new SqlConnection(connectionString))
{
    SqlCommand cmd = new SqlCommand("InsertMultipleProducts", conn);
    cmd.CommandType = CommandType.StoredProcedure;

    // Add table-valued parameter
    SqlParameter tvpParam = cmd.Parameters.AddWithValue(
        "@Products", productTable);
    tvpParam.SqlDbType = SqlDbType.Structured;
    tvpParam.TypeName = "ProductTableType";

    conn.Open();
    cmd.ExecuteNonQuery();

    Console.WriteLine($"Inserted {products.Count} products");
}

// Complex example with multiple operations
public class OrderItem
{
    public string ProductName { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
}

public int CreateOrderWithItems(int customerId, List<OrderItem> items)
{
    DataTable itemsTable = new DataTable();
    itemsTable.Columns.Add("ProductName", typeof(string));
    itemsTable.Columns.Add("Quantity", typeof(int));
    itemsTable.Columns.Add("UnitPrice", typeof(decimal));

    foreach (var item in items)
    {
        itemsTable.Rows.Add(item.ProductName, item.Quantity, item.UnitPrice);
    }

    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        SqlCommand cmd = new SqlCommand("CreateOrderWithItems", conn);
        cmd.CommandType = CommandType.StoredProcedure;

        cmd.Parameters.AddWithValue("@CustomerId", customerId);

        SqlParameter tvpParam = cmd.Parameters.AddWithValue("@OrderItems", itemsTable);
        tvpParam.SqlDbType = SqlDbType.Structured;
        tvpParam.TypeName = "OrderItemTableType";

        SqlParameter outputParam = new SqlParameter("@NewOrderID", SqlDbType.Int);
```

```
        outputParam.Direction = ParameterDirection.Output;
        cmd.Parameters.Add(outputParam);

        conn.Open();
        cmd.ExecuteNonQuery();

        return (int)outputParam.Value;
    }
}
```

Practical Coding Examples

16. Complete CRUD Operations with [ADO.NET](#)

```
public class ProductRepository
{
    private readonly string connectionString;

    public ProductRepository(string connString)
    {
        connectionString = connString;
    }

    // CREATE
    public int Insert(Product product)
    {
        using (SqlConnection conn = new SqlConnection(connectionString))
        {
            string query = @"INSERT INTO Products (ProductName, Price, CategoryID)
                              VALUES (@name, @price, @categoryId);
                              SELECT CAST(SCOPE_IDENTITY() as int)";

            SqlCommand cmd = new SqlCommand(query, conn);
            cmd.Parameters.AddWithValue("@name", product.ProductName);
            cmd.Parameters.AddWithValue("@price", product.Price);
            cmd.Parameters.AddWithValue("@categoryId", product.CategoryID);

            conn.Open();
            int newId = (int)cmd.ExecuteScalar();
            return newId;
        }
    }

    // READ - Get by ID
    public Product GetById(int id)
    {
        using (SqlConnection conn = new SqlConnection(connectionString))
        {
            string query = "SELECT * FROM Products WHERE ProductID = @id";

            SqlCommand cmd = new SqlCommand(query, conn);
            cmd.Parameters.AddWithValue("@id", id);

            conn.Open();
            SqlDataReader reader = cmd.ExecuteReader();

            if (reader.Read())
            {
                return new Product
            }
        }
    }
}
```

```
        {
            ProductID = reader.GetInt32(0),
            ProductName = reader.GetString(1),
            Price = reader.GetDecimal(2),
            CategoryID = reader.IsDBNull(3) ? (int?)null : reader.GetInt32(3)
        };
    }

    return null;
}

// READ - Get All
public List<Product> GetAll()
{
    List<Product> products = new List<Product>();

    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        string query = "SELECT * FROM Products ORDER BY ProductName";

        SqlCommand cmd = new SqlCommand(query, conn);

        conn.Open();
        SqlDataReader reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            products.Add(new Product
            {
                ProductID = reader.GetInt32(0),
                ProductName = reader.GetString(1),
                Price = reader.GetDecimal(2),
                CategoryID = reader.IsDBNull(3) ? (int?)null : reader.GetInt32(3)
            });
        }
    }

    return products;
}

// UPDATE
public bool Update(Product product)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        string query = @"UPDATE Products
                        SET ProductName = @name,
                          Price = @price,
                          CategoryID = @categoryId
                        WHERE ProductID = @id";

        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddWithValue("@name", product.ProductName);
        cmd.Parameters.AddWithValue("@price", product.Price);
        cmd.Parameters.AddWithValue("@categoryId",
            product.CategoryID.HasValue ? (object)product.CategoryID : DBNull.Value);
        cmd.Parameters.AddWithValue("@id", product.ProductID);

        conn.Open();
        int rowsAffected = cmd.ExecuteNonQuery();
        return rowsAffected > 0;
    }
}
```

```
    }  
}  
  
// DELETE  
public bool Delete(int id)  
{  
    using (SqlConnection conn = new SqlConnection(connectionString))  
    {  
        string query = "DELETE FROM Products WHERE ProductID = @id";  
  
        SqlCommand cmd = new SqlCommand(query, conn);  
        cmd.Parameters.AddWithValue("@id", id);  
  
        conn.Open();  
        int rowsAffected = cmd.ExecuteNonQuery();  
        return rowsAffected > 0;  
    }  
}  
  
// SEARCH  
public List<Product> Search(string searchTerm, decimal? minPrice, decimal? maxPrice)  
{  
    List<Product> products = new List<Product>();  
  
    using (SqlConnection conn = new SqlConnection(connectionString))  
    {  
        StringBuilder query = new StringBuilder("SELECT * FROM Products WHERE 1=1");  
  
        SqlCommand cmd = new SqlCommand();  
        cmd.Connection = conn;  
  
        if (!string.IsNullOrEmpty(searchTerm))  
        {  
            query.Append(" AND ProductName LIKE @search");  
            cmd.Parameters.AddWithValue("@search", "%" + searchTerm + "%");  
        }  
  
        if (minPrice.HasValue)  
        {  
            query.Append(" AND Price >= @minPrice");  
            cmd.Parameters.AddWithValue("@minPrice", minPrice.Value);  
        }  
  
        if (maxPrice.HasValue)  
        {  
            query.Append(" AND Price <= @maxPrice");  
            cmd.Parameters.AddWithValue("@maxPrice", maxPrice.Value);  
        }  
  
        cmd.CommandText = query.ToString();  
  
        conn.Open();  
        SqlDataReader reader = cmd.ExecuteReader();  
  
        while (reader.Read())  
        {  
            products.Add(new Product  
            {  
                ProductID = reader.GetInt32(0),  
                ProductName = reader.GetString(1),  
                Price = reader.GetDecimal(2),  
                CategoryID = reader.IsDBNull(3) ? (int?)null : reader.GetInt32(3)  
            });  
        }  
    }  
}
```

```
        }));
    }
}

return products;
}
}

// Product Model
public class Product
{
    public int ProductID { get; set; }
    public string ProductName { get; set; }
    public decimal Price { get; set; }
    public int? CategoryID { get; set; }
}
```

17. Data Access Layer with Error Handling

```
public class DataAccessLayer
{
    private readonly string connectionString;
    private readonly ILogger logger;

    public DataAccessLayer(string connString, ILogger log)
    {
        connectionString = connString;
        logger = log;
    }

    // Generic Execute method with error handling
    public T Execute<T>(Func<SqlConnection, T> operation)
    {
        try
        {
            using (SqlConnection conn = new SqlConnection(connectionString))
            {
                conn.Open();
                return operation(conn);
            }
        }
        catch (SqlException ex)
        {
            logger.LogError($"SQL Error: {ex.Message}");
            logger.LogError($"Error Number: {ex.Number}");
            logger.LogError($"Line Number: {ex.LineNumber}");

            // Handle specific SQL errors
            switch (ex.Number)
            {
                case 2627: // Duplicate key
                    throw new DuplicateRecordException("Record already exists", ex);
                case 547: // Foreign key violation
                    throw new ForeignKeyException("Foreign key constraint violation", ex);
                case -1: // Timeout
                    throw new TimeoutException("Database timeout occurred", ex);
                default:
                    throw;
            }
        }
        catch (Exception ex)
    }
```

```

        {
            logger.LogError($"Unexpected error: {ex.Message}");
            throw;
        }
    }

    // Execute with retry logic
    public T ExecuteWithRetry<T>(Func<SqlConnection, T> operation, int maxRetries = 3)
    {
        int retryCount = 0;

        while (true)
        {
            try
            {
                return Execute(operation);
            }
            catch (SqlException ex) when (retryCount < maxRetries && IsTransientError(ex))
            {
                retryCount++;
                logger.LogWarning($"Transient error occurred. Retry {retryCount}/{maxRetries}");
                Thread.Sleep(TimeSpan.FromSeconds(Math.Pow(2, retryCount))); // Exponential backoff
            }
        }
    }

    private bool IsTransientError(SqlException ex)
    {
        // Transient error codes
        int[] transientErrors = { -1, -2, 1205, 4060, 40197, 40501, 40613, 49918, 49919, 49920 };
        return transientErrors.Contains(ex. Number);
    }
}

// Custom exceptions
public class DuplicateRecordException : Exception
{
    public DuplicateRecordException(string message, Exception inner)
        : base(message, inner) { }
}

public class ForeignKeyException : Exception
{
    public ForeignKeyException(string message, Exception inner)
        : base(message, inner) { }
}

```

18. Generic Repository Pattern

```

public interface IRepository<T> where T : class
{
    T GetById(int id);
    IEnumerable<T> GetAll();
    int Insert(T entity);
    bool Update(T entity);
    bool Delete(int id);
}

public class GenericRepository<T> : IRepository<T> where T : class, new()
{
    private readonly string connectionString;

```

```
private readonly string tableName;

public GenericRepository(string connString, string table)
{
    connectionString = connString;
    tableName = table;
}

public T GetById(int id)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        string query = $"SELECT * FROM {tableName} WHERE {tableName}ID = @id";
        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddWithValue("@id", id);

        conn.Open();
        SqlDataReader reader = cmd.ExecuteReader();

        if (reader.Read())
        {
            return MapToEntity(reader);
        }

        return null;
    }
}

public IEnumerable<T> GetAll()
{
    List<T> entities = new List<T>();

    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        string query = $"SELECT * FROM {tableName}";
        SqlCommand cmd = new SqlCommand(query, conn);

        conn.Open();
        SqlDataReader reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            entities.Add(MapToEntity(reader));
        }
    }

    return entities;
}

private T MapToEntity(SqlDataReader reader)
{
    T entity = new T();
    Type type = typeof(T);

    foreach (var property in type.GetProperties())
    {
        if (! reader.HasColumn(property.Name))
            continue;

        object value = reader[property.Name];

        if (value != DBNull.Value)
```

```
        {
            property.SetValue(entity, value);
        }
    }

    return entity;
}

public int Insert(T entity)
{
    // Implementation would use reflection to build INSERT query
    throw new NotImplementedException();
}

public bool Update(T entity)
{
    // Implementation would use reflection to build UPDATE query
    throw new NotImplementedException();
}

public bool Delete(int id)
{
    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        string query = $"DELETE FROM {tableName} WHERE {tableName}ID = @id";
        SqlCommand cmd = new SqlCommand(query, conn);
        cmd.Parameters.AddWithValue("@id", id);

        conn.Open();
        return cmd.ExecuteNonQuery() > 0;
    }
}

// Extension method for SqlDataReader
public static class SqlDataReaderExtensions
{
    public static bool HasColumn(this SqlDataReader reader, string columnName)
    {
        for (int i = 0; i < reader.FieldCount; i++)
        {
            if (reader.GetName(i).Equals(columnName, StringComparison.OrdinalIgnoreCase))
                return true;
        }
        return false;
    }
}
```

Best Practices Summary

- 1. Always use parameterized queries**
- 2. Dispose connections properly (using statements)**
- 3. Use connection pooling**
- 4. Handle exceptions appropriately**

5. Use stored procedures for complex operations
 6. Implement transactions for multiple related operations
 7. Use async methods for better scalability
 8. Choose correct architecture (connected vs disconnected)
 9. Implement proper logging
 10. Follow repository pattern for better maintainability
-

Common Interview Scenarios

****Scenario 1: When to use DataReader vs DataSet? ****

- Use DataReader for: Read-only, forward-only, large datasets, performance-critical operations
- Use DataSet for: Offline data manipulation, multiple tables with relationships, binding to UI controls

Scenario 2: How to improve [ADO.NET](#) performance?

- Use connection pooling
- Use stored procedures
- Use parameterized queries
- Close connections immediately
- Use appropriate data types
- Use DataReader for read-only operations
- Implement caching
- Use async operations

Scenario 3: How to handle database deadlocks?

- Use appropriate isolation levels
 - Keep transactions short
 - Access resources in same order
 - Implement retry logic
 - Use proper indexing
 - Monitor using SQL Profiler
-

Additional Resources

- Microsoft [ADO.NET](#) Documentation
- SQL Server Best Practices
- Design Patterns for Data Access
- Entity Framework (Modern Alternative)
- Dapper (Micro-ORM Alternative)